



Services, Libraries, Gradle plugin

DATE 2023

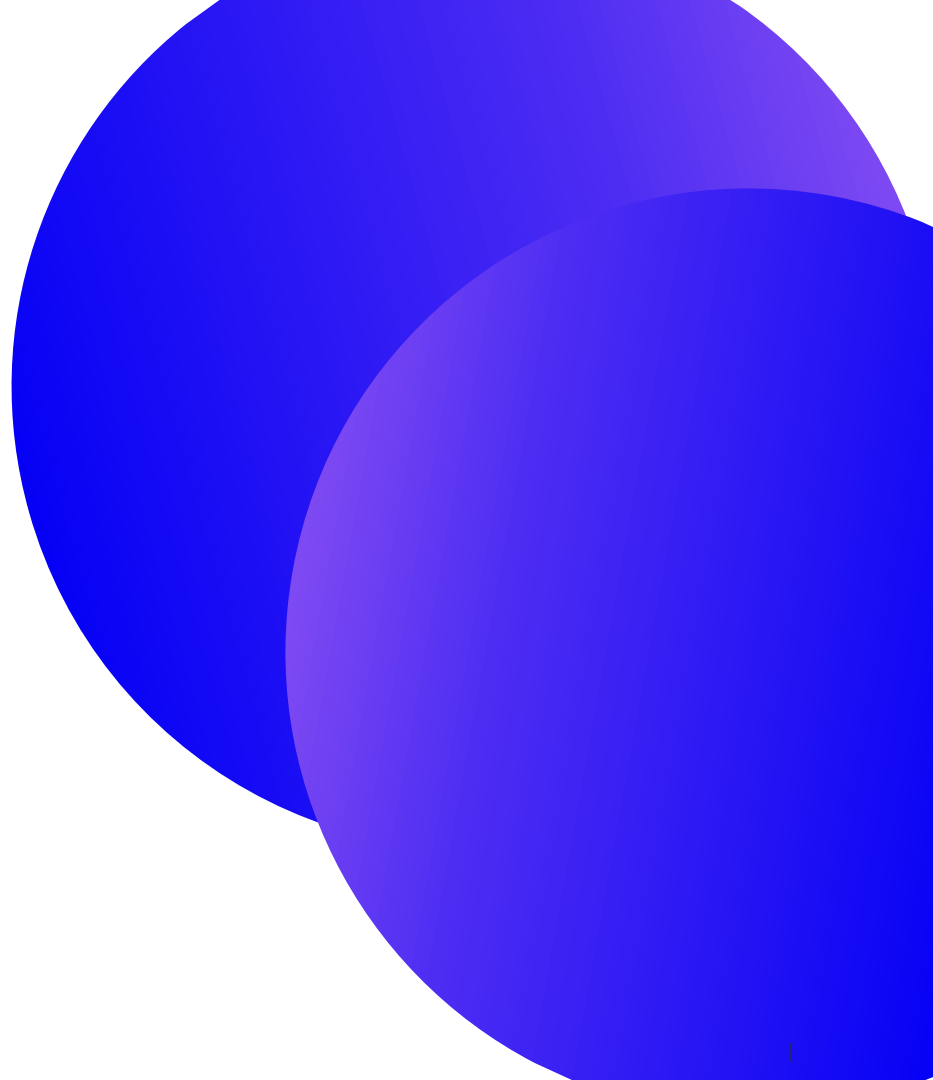
Lukas Prokop, Simona Kurnavova



Agenda

- Services
- Libraries
- Gradle plugins
- Build process

Services



Services

- [Documentation](#)
- Long running operation in background
- Not bound with UI
- Can expose API for other applications
- **By default runs on UI thread**

Services

- **Types:**
 - Foreground
 - Background (limited since Oreo)
 - Started
 - Bound

Services

- Needs to be declared in manifest:



```
<manifest ... >
  ...
  <application ... >
    <service android:name=".ExampleService" />
    ...
  </application>
</manifest>
```

Started service

- Started by another component calling `startService()`
- When started, it's lifecycle is independent from the caller
- Does not return result to caller
- *Example usage:* started from Activity in order to save data to database.

Started service: Starting



Starting of Service

```
context.startService(Intent(context, HelloService::class.java))
```



Must implement onStartCommand

```
class ExampleService : Service() {  
    ...  
  
    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {  
        // The service is starting, due to a call to startService()  
        return startMode  
    }  
  
    ...  
}
```


Started service: Stopping

- **System will not stop started service**
- Service can stop itself: `Service#stopSelf()`
- Other component can stop the service: `Context#stopService()`

Bound service

- Service is bound when another component calls `bindService()`
- Offers client-server interface for communication
- Lightweight RPC communication
- *Example usage:* Music player service with controls in Activity.

Bound service: Binding

- Component binds to Service by calling:

```
Context#bindService(service: Intent  
                    conn: ServiceConnection,  
                    flags: Int): Boolean
```

- Override `Service#onBind(intent: Intent): IBinder?`
- Service returns `IBinder` object for interaction:
 - Remotable object for communication with bounded service
 - Can be defined by AIDL (Android interface definition language)

Bound service: Implementation

```
Bound Service implementation

class LocalService : Service() {

    private val binder = LocalBinder() // Binder given to clients.

    /** Method for clients. */
    val randomNumber: Int
        get() = Random().nextInt(100) // Generate random number

    /**
     * Class used for the client Binder.
     */
    inner class LocalBinder : Binder() {
        // Return this instance of LocalService so clients can call public methods.
        fun getService(): LocalService = this@LocalService
    }

    override fun onBind(intent: Intent): IBinder {
        return binder
    }
}
```

Bound services: Unbind

- Clients call `Context#unbindService(conn: ServiceConnection)`
- System destroys service, when all clients unbind from it

Bound services: ServiceConnection

- Clients need to implement ServiceConnection callbacks

```
ServiceConnection implementation

private val connection = object : ServiceConnection {

    override fun onServiceConnected(className: ComponentName, service: IBinder) {
        // Connection with the service has been established.
        val binder = service as LocalService.LocalBinder
        val service = binder.getService()
        val randomNum = service.randomNumber
        ...
    }

    override fun onServiceDisconnected(arg0: ComponentName) {
        // Service was disconnected.
        // Process hosting service crashed or been killed
        // Service connection remain active (onServiceConnected can be called again)
    }

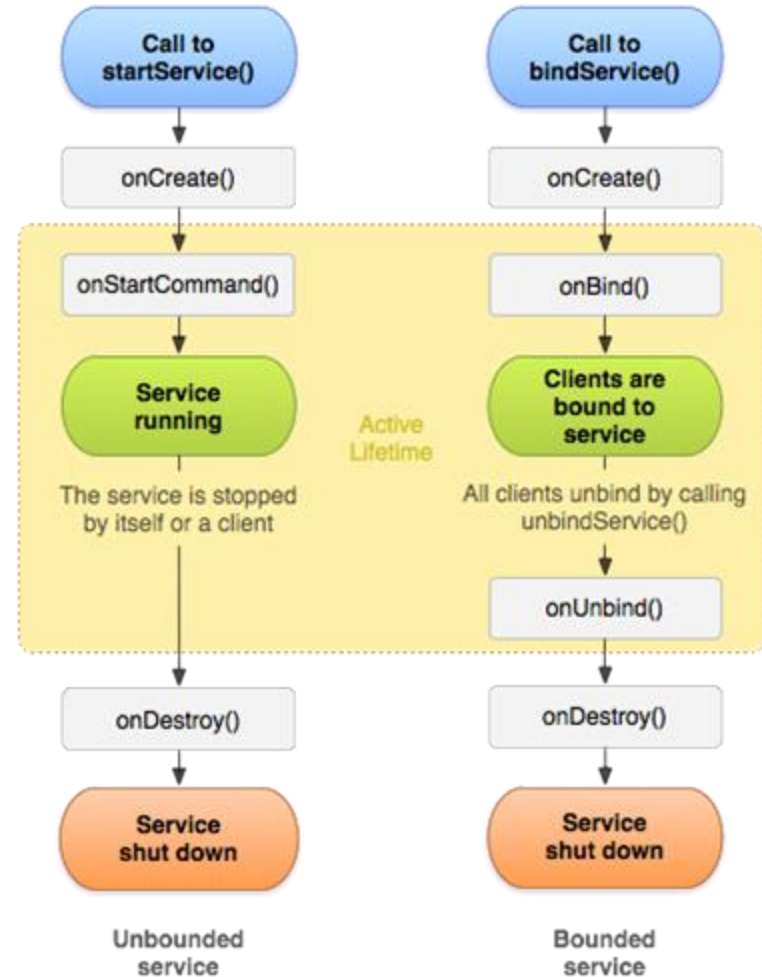
    override fun onBindingDied(name: ComponentName?) { // Optional impl
        // Binding is dead
        // Can happen during app update
        // Unbind and rebind
    }

    override fun onNullBinding(name: ComponentName?) { // Optional impl
        // Service#onBind returns null
        // Unbinding is still required
    }
}
```

Service lifecycle

onCreate()

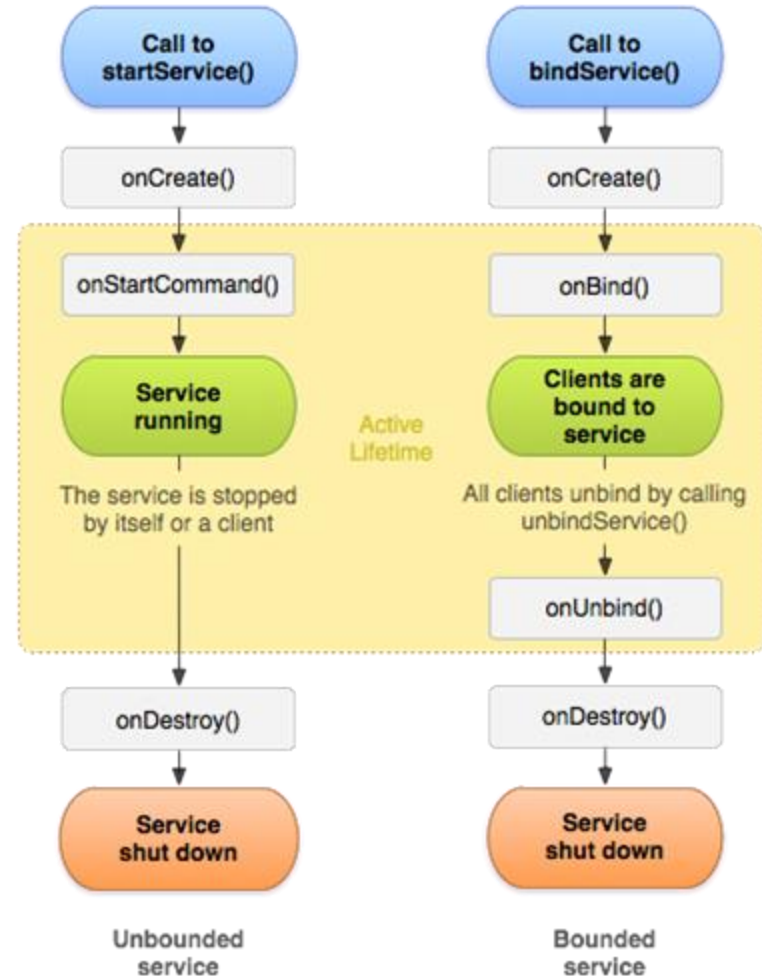
- Called when the service is being created (after first call of `startService()` or `bindService()`)



Service lifecycle

onStartCommand()

- Called when `startService()` is called, delivers starting intent
- Returned value specify behaviour when it's killed by system
 - **START_STICKY** - don't retain intent, later when system recreate service null intent is delivered (explicitly started/stopped services)
 - **START_NOT_STICKY** - if there is no start intent, take service out of the started state. Service is not recreated.
 - **START_REDELIVER_INTENT** - last delivered intent will be redelivered, pending intent delivered at the point of restart



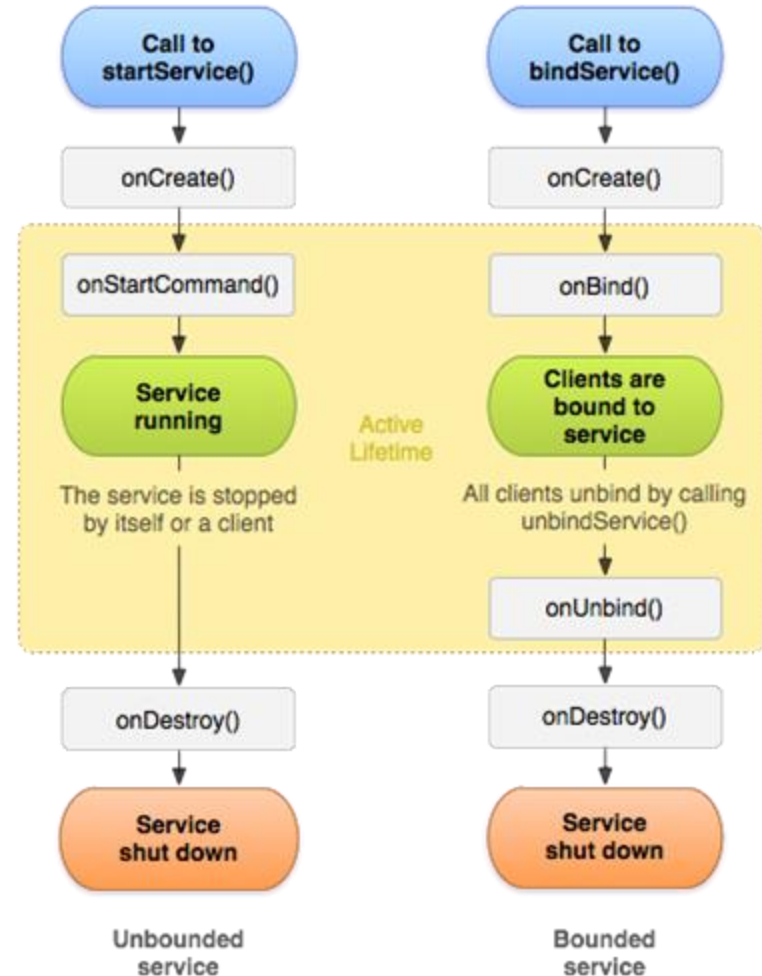
Service - lifecycle

onBind()

- When another component binds to service
- Returns Binder object for communication

onUnbind()

- When all clients disconnected from interface published by service
- Returns true when onRebind should be called when new clients bind to service, otherwise onBind will be called



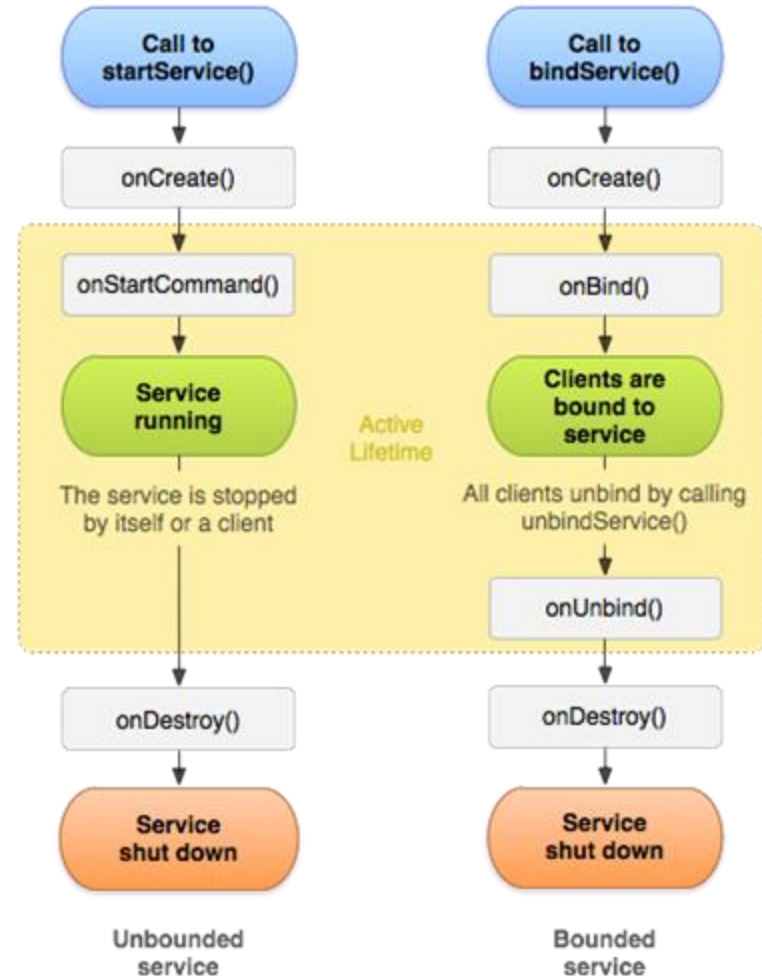
Service - lifecycle

onRebind()

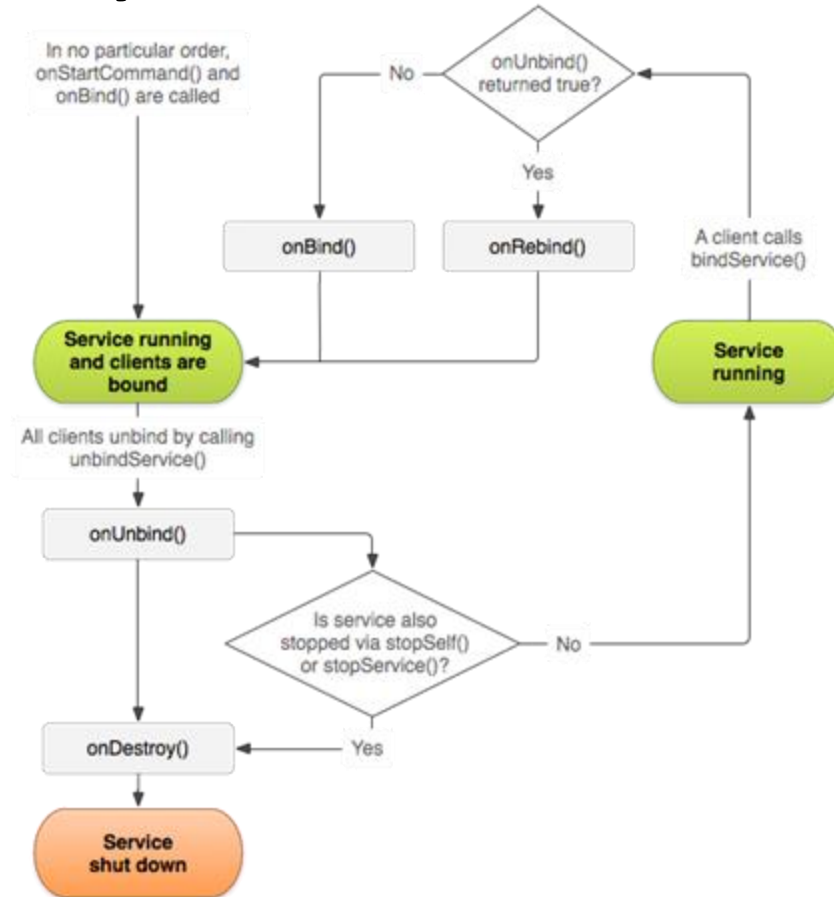
- Called when new clients are connected, after notification about disconnecting all client in its onUnbind

onDestroy()

- Called by system to notify a Service that it is no longer used and is being removed.
- Cleanup receivers, threads..



Bound Service lifecycle

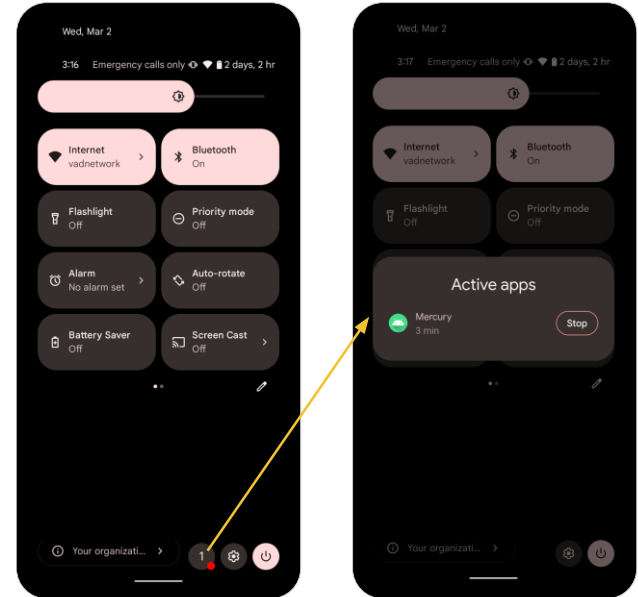


Background service

- On background by default
- Strongly limited since Android Oreo (API 26)
 - Not possible to start background service when app is not on the foreground
 - [Details in documentation](#)
- *Example usage:* Sync against backend
- Alternatively use WorkManager

Foreground service

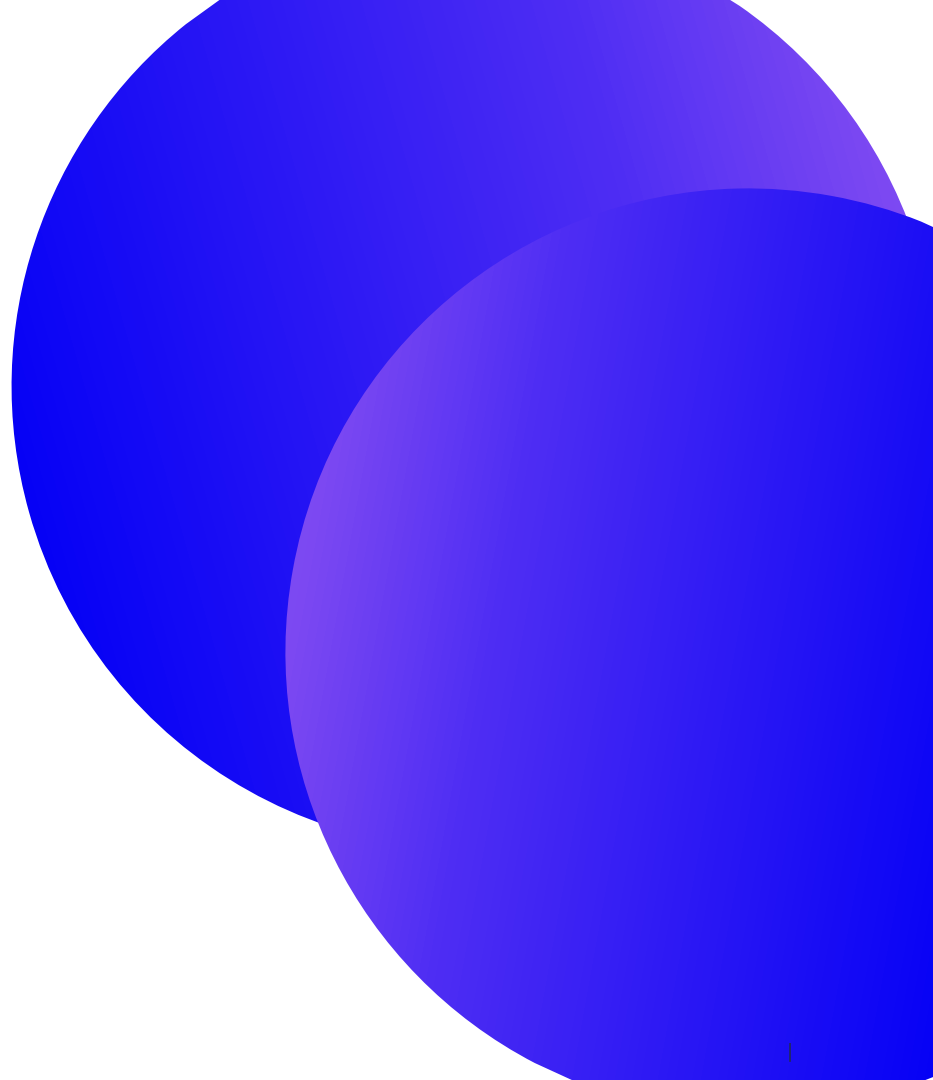
- Service process has higher priority
- User is actively aware of it – requires **permanent notification**
 - Notification shown under Ongoing header
 - Lower than Android 13: notification cannot be dismissed
 - Android 13 and higher: notification dismissible by default
- System not likely to kill foreground services
- Apps targeting Android 9 (API 28) or higher must define
 - FOREGROUND_SERVICE permission (normal permission)
- *Example usage*: Downloading files
- Alternatively use foreground Workers



Foreground service: Starting

- Starting foreground service: `Context#startForegroundService(Intent)`
- Make service foreground: `Service#startForeground(int, Notification)`
 - Needs to be called within 5s of being started, otherwise Service will be stopped
- Remove from foreground: `stopForeground()`

Libraries, gradle
plugins, etc...

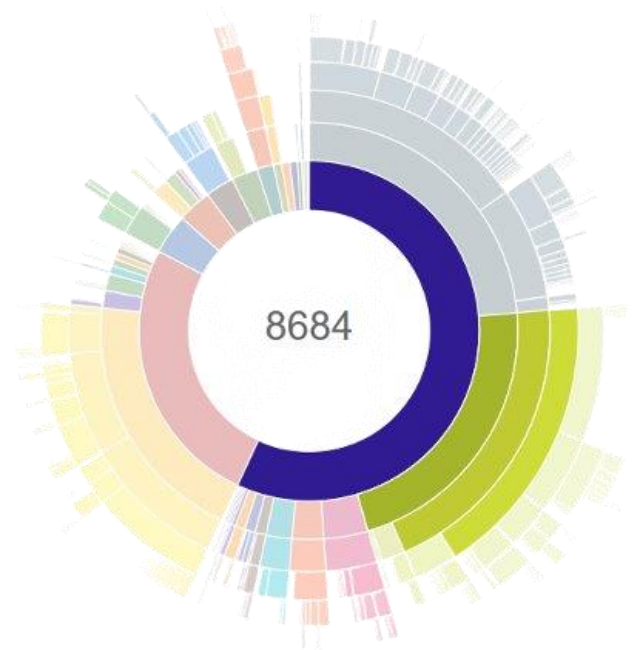


Android jetpack

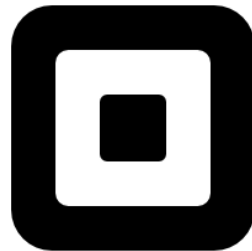
- Set of libraries from Google
- [Documentation](#)
- Groups
 - **Foundation:** AppCompatActivity, Android KTX, Multidex, Test
 - **Architecture:** Data binding, Lifecycles, LiveData, Navigation, Paging, Room, ViewModel, WorkManager
 - **Behavior:** Download manager, Media & playback, Notifications, Permissions, Preferences, Sharing, Slices
 - **UI:** Animations & transitions, Auto, Emoji, Fragment, Layout, Palette, TV, Wear OS

DexCount plugin

- Computes methods count in APK
- Visualize count in a nice chart
- <https://github.com/KeepSafe/dexcount-gradle-plugin>

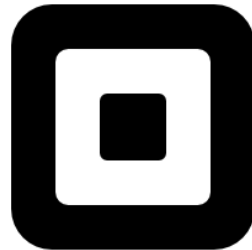


Retrofit



- A type-safe HTTP client for Android and Java
- Simplify communication with some API service
- Configurable
 - OkHTTP 3 client - compression, timeouts
 - Supports multiple convertors
 - Gson
 - Jackson
 - Moshi
 - Protobuf
 - Wire
- <https://square.github.io/retrofit/>

OkHttp



- An HTTP & HTTP/2 client for Android and Java applications
- Supports sync/async calls
- Supports multiple addresses per URL (Load balancing, failover)
- <http://square.github.io/okhttp/>

Ktor

- Multiplatform http client/server from JetBrains
- Build on top of Kotlin and Coroutines
- <https://ktor.io>



Dependency Injection frameworks

- **Why:**
 - Decouple code
 - Better testing
- Frameworks:
 - Dagger (Java, Kotlin, Android)
 - Hilt (Android)
 - Koin (Kotlin, multiplatform)
 - KodeinDI (Kotlin, multiplatform)

Flipper

- Debug platform by Facebook
- Inspect
 - ViewHierarchy
 - Database
 - Shared preferences
 - Network traffic
- <https://fbflipper.com/>



LeakCanary

- Helps with finding memory leaks
- <https://github.com/square/leakcanary>



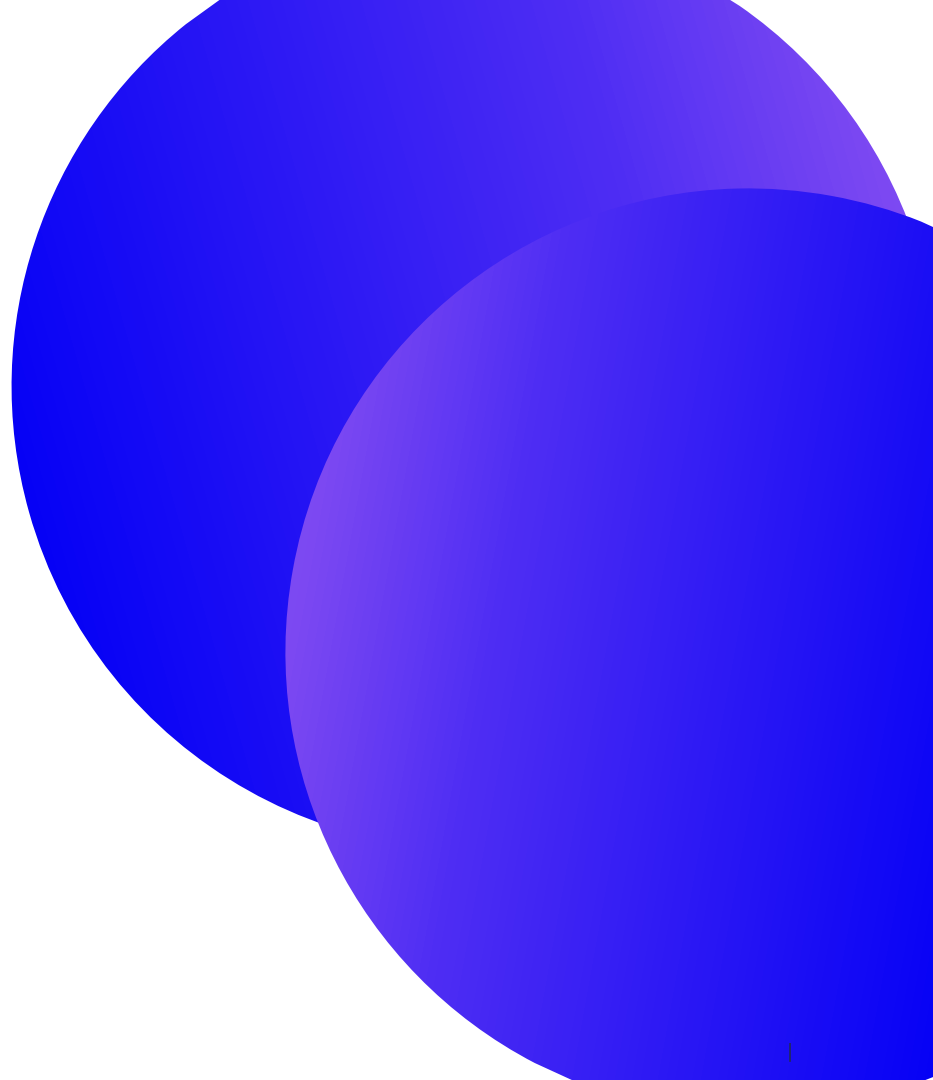
Kotlinx.serialization

- Data serialization/deserialization
 - Json
 - Protocol buffers
- <https://kotlinlang.org/docs/serialization.html>

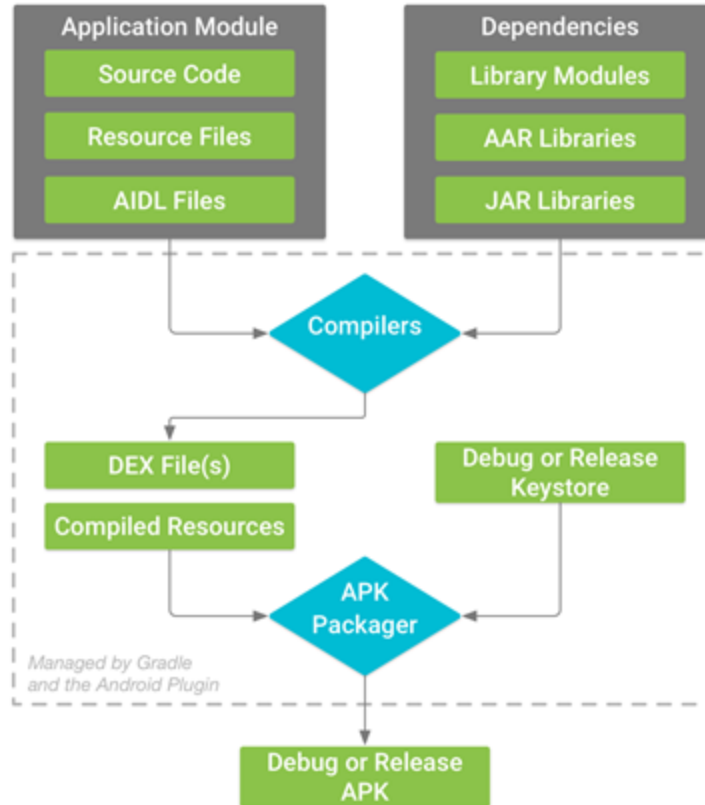
Demo

- Implement network repository

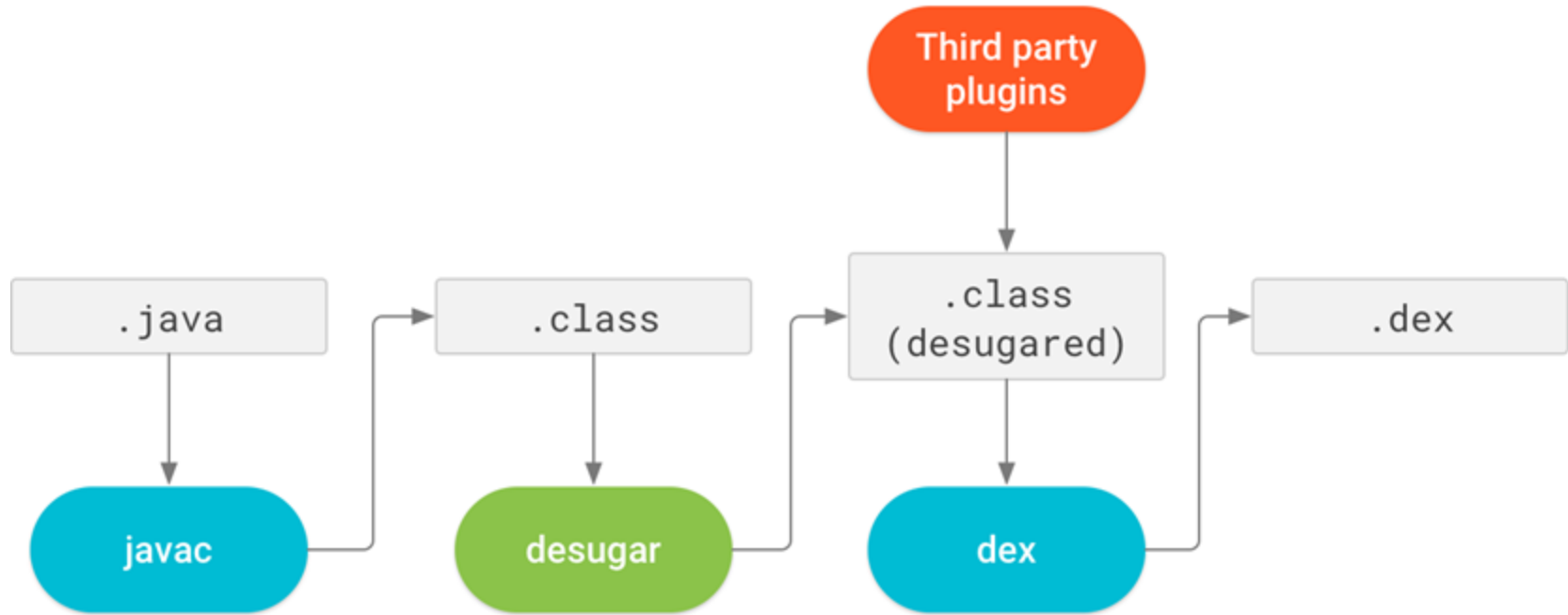
Build process and APK structure



Build process



Support java 8 features (d8)



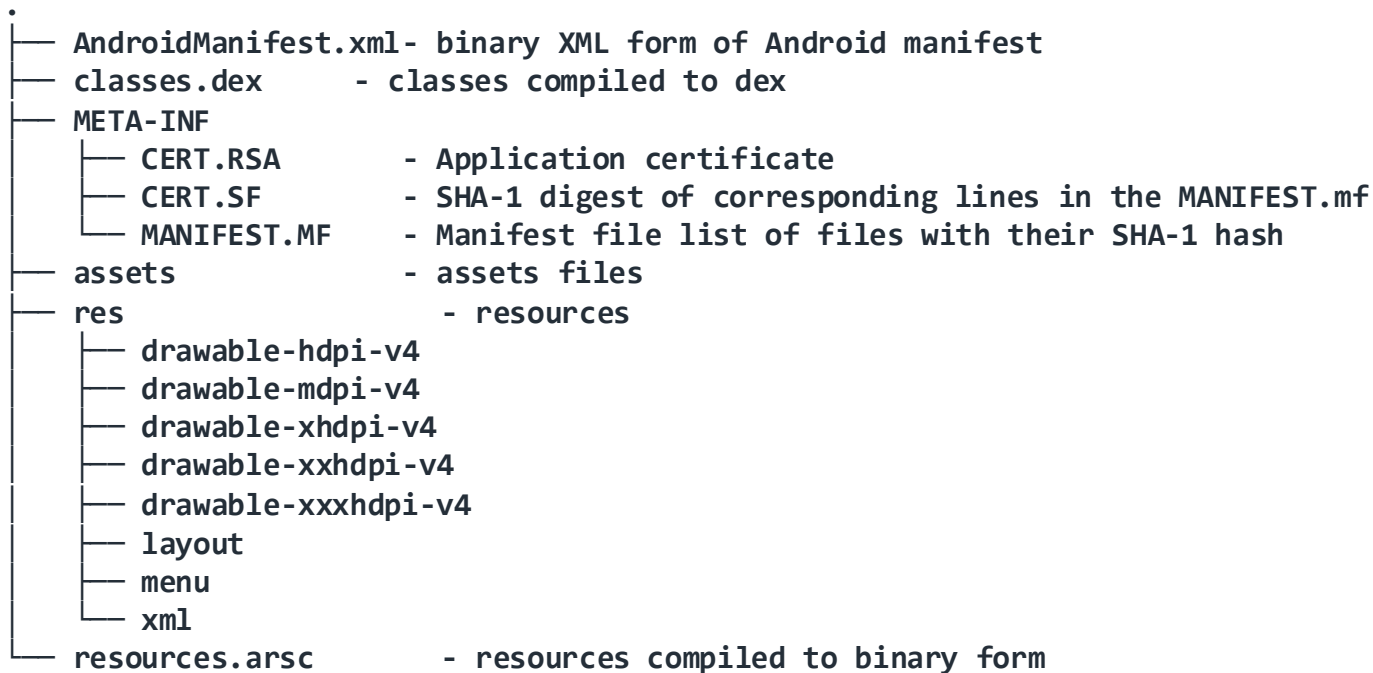
Building android apps limitations

- **Dex limit 64k methods**
 - Shrink unused methods and classes (libraries)
 - You need to specify what is entry point, build dependency graph. Classes which are not part of graph are removed, unused methods as well
 - When you work on library, provide proguard rules together with library
 - Sometimes is better copy some classes from library into application
 - for example guava library
- **Google doesn't allow code side load**

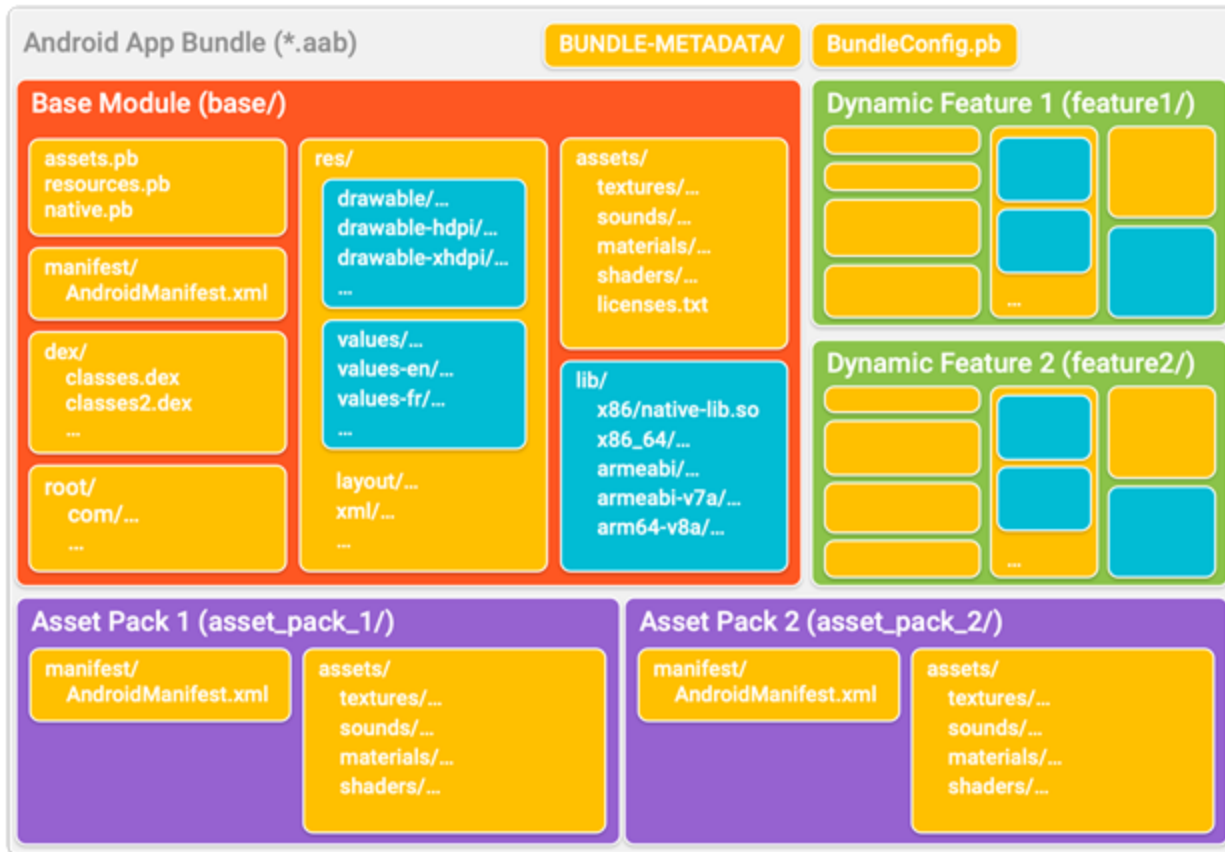
Multidex

- Native support since API-21 for older version support library
- Try to avoid using multidex, it slows down application start
- Splits classes to multiple dex files
- on API $\geq$ 21 dex files are converted to single .oat file (ART runtime)
- Main dex file loaded when app is started
- Loading of additional dex files is performed during initialization
- Dex files are in app folder
- <https://www.blackhat.com/docs/ldn-15/materials/london-15-Welton-Abusing-Android-Apps-And-Gaining-Remote-Code-Execution.pdf>

Apk structure



AppBundle



Proguard/R8

- **Shrink** - smaller code, faster build
- **Optimize** - faster code, removing static conditions
- **Obfuscate** - make it harder to read

Decompile apk

- **Unzip the apk**
- **Dex2Jar to convert classes.dex to jar archive**
 - <https://github.com/pxb1988/dex2jar>
- **jd-gui to view decompiled classes**
 - <http://jd.benow.ca/>
- **BytecodeViewer**
 - <https://bytecodeviewer.com/>
- **Android studio apk analyzer**
 - Easy to check resources
 - Compare multiple apk